



Исходные данные

Команда разрабатывает программный продукт и планирует использовать несколько AI-моделей для разных типов задач. Использовать одну модель для всех операций не требуется. Для каждого сценария можно выбрать ту модель, характеристики которой лучше соответствуют условиям задачи.

Для сравнения доступны четыре условные модели.

Характеристика	Model A	Model B	Model C	Model D
Средняя задержка ответа	0,4–1 с	2–5 с	8–20 с	3–8 с
Качество работы с простым кодом	высокое	высокое	очень высокое	высокое
Качество сложного анализа кода	среднее	высокое	очень высокое	высокое
Контекстное окно	32 000 токенов	128 000 токенов	256 000 токенов	1 000 000 токенов
Относительная стоимость	низкая	средняя	высокая	высокая
Работа с инструментами	базовая	поддерживается	расширенная	поддерживается
Мультимодальность	нет	есть	есть	есть
Возможность локального запуска	да	нет	нет	нет

Стоимость в таблице указана относительно остальных моделей и используется только для выполнения данной практической работы.

Кейс 1. Быстрый автокомплит

Команда хочет добавить AI-автодополнение в редактор кода. Разработчик пишет программу, а модель должна практически сразу предлагать продолжение текущей строки или небольшого блока.

Типичный запрос содержит текущий файл и небольшой объем окружающего кода. В большинстве случаев размер передаваемого контекста составляет от 1 000 до 8 000 токенов.

Например, разработчик начинает писать функцию:

```
def calculate_total(items):  
    total = 0  
  
    for item in items:
```

Система должна предложить вероятное продолжение.

Такие запросы будут выполняться постоянно во время рабочего дня. Один разработчик может обращаться к модели сотни раз.

Если подсказка появляется через несколько секунд, разработчик часто успевает написать продолжение самостоятельно и предложение теряет ценность.

Большое контекстное окно для большинства запросов не требуется. Мультимодальность также не используется.

Компания планирует подключить функцию автокомплита для 60 разработчиков.

Необходимо выбрать модель для данного сценария и обосновать выбор с точки зрения latency, качества кода, стоимости, контекстного окна и частоты использования.

Также необходимо определить, какие характеристики моделей в этом сценарии имеют низкий приоритет и за какие возможности компании нет смысла переплачивать.

Кейс 2. Глубокий рефакторинг

Команда поддерживает backend интернет-магазина. Один из модулей создавался несколько лет назад и постепенно стал сложным для сопровождения.

В модуле находится около 35 Python-файлов. Общий объём релевантного кода составляет примерно 80 000 токенов.

Необходимо провести глубокий рефакторинг системы расчёта заказов.

Существующая логика распределена между сервисами обработки заказа, расчёта стоимости, скидок, промокодов и доставки. Несколько функций дублируют вычисления. Некоторые методы имеют большое количество условий. Между несколькими компонентами существуют зависимости, которые нельзя нарушить.

Часть методов вызывается следующим образом:

```
order_service.calculate(order)
discount_service.apply(order)
delivery_service.calculate(order)
```

После рефакторинга внешнее поведение API должно сохраниться. Изменения не должны нарушить существующие тесты.

В данном сценарии разработчик не ожидает мгновенного ответа. Получение качественного результата через несколько секунд или даже десятки секунд считается допустимым.

Количество таких запросов значительно меньше, чем при автокомплите. Команда выполняет глубокий рефакторинг отдельных компонентов несколько раз в месяц.

Особенно важно, чтобы модель могла учитывать взаимосвязи между большим количеством файлов, обнаруживать побочные эффекты изменений и качественно работать со сложным программным кодом.

Предполагается, что AI-система также сможет использовать инструменты для чтения файлов проекта и запуска тестов.

Необходимо выбрать модель для данного сценария и объяснить, какие характеристики становятся определяющими.

Следует отдельно рассмотреть, насколько оправдано использование наиболее дорогой модели и будет ли максимальное контекстное окно обязательным именно для этого проекта.

Кейс 3. Анализ большой кодовой базы

Компания приобрела программный продукт другой команды. Перед началом активной разработки необходимо провести первоначальный анализ проекта.

Репозиторий содержит около 7 000 файлов. В нём присутствуют backend, frontend, SQL-миграции, конфигурационные файлы, тесты и документация.

Общий объём исходных материалов значительно превышает контекстное окно любой из рассматриваемых моделей.

Структура проекта частично выглядит следующим образом:

```
project/  
  backend/  
  frontend/  
  database/  
  migrations/  
  tests/  
  docs/  
  scripts/  
  config/
```

Команда хочет получить ответы на вопросы о структуре проекта, связях между основными модулями, использовании конкретных API и возможных зависимостях между компонентами.

Например, разработчик может поставить задачу:

Определи, какие части проекта используют старую версию API платежного сервиса, и найди компоненты, которые потенциально потребуется изменить при миграции.

Для выполнения задачи AI-системе может потребоваться анализировать большое количество файлов и несколько раз обращаться к различным частям репозитория.

Мгновенный ответ не является главным требованием. Допустимо потратить больше времени, если это повысит полноту анализа.

При этом передавать всю кодовую базу в каждый запрос дорого и технически не всегда возможно.

В проекте предполагается использовать поиск релевантных файлов и передавать модели только те участки кодовой базы, которые относятся к текущему вопросу.

Необходимо определить подходящую модель для данного сценария.

При принятии решения необходимо учитывать контекстное окно, качество анализа кода, стоимость обработки большого объёма информации и поддержку инструментов.

Также необходимо решить, является ли модель с самым большим контекстным окном автоматически лучшим вариантом или часть задачи рациональнее решить за счёт правильного отбора контекста.

Материал для сравнения результатов

После выбора моделей необходимо зафиксировать решение для каждого из трёх сценариев в едином формате.

Сценарий	Выбранная модель	Главный критерий	Вторичный критерий	Допустимый компромисс	Причина отказа от остальных вариантов
Быстрый авто-комплит					
Глубокий рефакторинг					
Анализ большой кодовой базы					

В конце практической работы необходимо определить, имеет ли смысл использовать одну универсальную модель во всех трёх сценариях или разделение задач между несколькими моделями позволит получить более рациональное соотношение скорости, качества и стоимости.
